Stochastic Gradient Descent and Variance Reduction Methods

Sarmad Ahmed

Gradient Descent Limitations

- In machine learning models are trained by minimizing a loss function over large scale datasets
- Loss for a single point is $f_i(w)$ which measures how well the model correctly predicts the true label for data point i.
- Objective function: Minimizing the average loss across all examples
  - minimize $f(w) = 1/n * \sum f_i(w)$ w/ respect to w
- Gradient descent computes the gradient using all data points at each iteration to update the weights, w.
  - $1/n * \sum \nabla f_i(w)$
- Computing the gradient for every single data point costs O(n) per iteration which makes it computationally infeasible for large scale amounts of data.
- This motivated the shift into more efficient optimization methods for machine learning.

Stochastic Gradient Descent

- SGD addresses this issue by replacing the computation of the gradient of the loss function at every data point by instead computing the gradient of the loss function at a randomly selected point at each iteration.
- Algorithm:
  1. Randomly select 1 point in dataset: point i
  2. Compute $\nabla f_i(w^{(k)})$: This is the gradient of the loss function evaluated at point i
  3. Update: $w^{(k+1)} = w^{(k)} - t^{(k)} \nabla f_i(w^{(k)})$
  4. Repeat for k iterations
- Cost per iteration reduced from O(n) to O(1)
- Assumes that the loss function is L smooth
  - Guarantees that the gradient does not change too abruptly
- Stochastic gradient introduces variance as it uses a single point at each iteration to estimate the true full gradient.
- Step sizes:
  - Fixed step size → does not lead to convergence due to noise of gradient at each iteration
  - Decreasing step size → Take big steps early in training, as algorithm approaches w* smaller steps taken so noise from gradient does not make the algorithm bounce around the optimal w
- This variance problem in the stochastic gradient led to the development of variance reduction methods SVRG, SAGA, and SARAH.

SVRG (Stochastic Variance Reduced Gradient)

- SVRG reduces variance by computing the full gradient (use all data points) using $w_{snapshot}$ once per epoch and using it to fix the noise in the stochastic gradient
- Algorithm:
  1. At the start of each epoch store the current weights as $w_{snapshot}$
  2. Compute the full gradient at snapshot weights: $1/n * \sum \nabla f_i(w_{snapshot})$
  3. **Inner Loop:** Select a random data point i
     - Compute gradient of loss at this point using current weights: $\nabla f_i(w^{(k)})$
     - Compute the gradient of loss at this point using snapshot weights: $\nabla f_i(w_{snapshot})$
  4. Compute the variance reduced gradient: $v = \nabla f_i(w^{(k)}) - \nabla f_i(w_{snapshot}) + 1/n * \sum \nabla f_i(w_{snapshot})$
  5. Update $w^{(k+1)} = w^{(k)} - tv$
  6. Repeat steps 3-5 for n iterations (n is # of samples) (**Inner Loop)**
  7. Update $w_{snapshot}$ to current weights and repeat from step 2
- Variance Reduction
  - $\nabla f_i(w^{(k)}) - \nabla f_i(w_{snapshot})$ reduces the noise from using one random point to estimate the full gradient
    - The gradient at data point i is evaluated at two nearby weights so the noise in both is identical and cancels out
  - As $w^{(k)}$ and $w_{snapshot}$ approach w^* : $w^{(k)}$ and $w_{snapshot}$ are close together so $\nabla f_i(w^{(k)}) - \nabla f_i(w_{snapshot})$ approaches 0 and we are left with $1/n * \sum \nabla f_i(w_{snapshot})$ which is the true full gradient without any noise.
- Note: SVRG uses step size $t = 1/10L$

SAGA (Stochastic Average Gradient)

- SAGA takes a different approach by maintaining a table of the most recent gradient computed for each point
- Algorithm:
  1. Compute and store gradient of loss for every data point: $\nabla f_i(w)$ for all i = 1…n
  2. Compute and store the avg of these gradients: $1/n * \sum \nabla f_i(w)$
  3. Select a random data point i
  4. Compute the gradient of the loss for this point: $\nabla f_i(w^{(k)})$
  5. Compute the variance reduced gradient: $v = \nabla f_i(w^{(k)})$ - grad_table[i] + $1/n * \sum \nabla f_i(w)$
  6. Update: $w^{(k+1)} = w^{(k)} - tv$
  7. Update gradient table
     a. Replace stored gradient for point i w/ newly computed gradient for point i
     b. Update the average
  8. Repeat steps 3 to 7 for n iterations (n is # of samples) - Completes one epoch
  9. Repeat for amount of epochs desired
- Variance Reduction:
  - As $w^{(k)}$ approaches w* weights stop changing so $\nabla fi(w(k))$ - grad_table[i] approaches 0 so we are left with the average of the gradient table an accurate estimate of the true gradient.
- Note: SAGA uses step size t = 1/3L

SARAH

- Algorithm
  1. Save the currents weights at start of epoch as $w^{(0)}$
  2. Compute the full gradient at w_0: $v_0 = 1/n * \sum \nabla f_i(w^{(0)})$
  3. Compute the first update using the full gradient: $w^{1)} = w^{(0)} - t*v^{0)}$
  4. Select a random data point i
     a. Compute the gradient at this point using the current weights: $\nabla f_i(w^{(k)})$
     b. Compute the gradient at this point using the previous weights: $\nabla f_i(w^{(k-1)})$
  5. Compute the gradient estimate $v^{(k)} = \nabla f_i(w^{(k)}) - \nabla f_i(w^{(k-1)}) + v^{(k-1)}$
  6. Update: $w^{(k+1)} = w^{(k)} - tv^{(k)}$
  7. Store old current weights as new previous weights $w^{(k-1)} = w^{(k)}$
  8. Repeat steps 4-7 for n, 2n, 3n,or 4n iterations
  9. At the end of inner loop pick a random iterate and use its weights as the new $w^{(0)}$ for next epoch
- Variance Reduction
  - $\nabla f_i(w^{(k)}) - \nabla f_i(w^{(k-1)})$ measures how much data point i's gradient changed between 2 iterations
  - As $w^{(k)}$ approaches w^* the updates between $w^{(k)}$ and $w^{(k-1)}$ get smaller so $\nabla f_i(w^{(k)}) - \nabla f_i(w^{(k-1)})$ approaches 0
  - We are left with $v^{(k-1)}$, the previous iterations full gradient estimate
- Note: SARAH uses step size $t = 1/L$

Setup

- Data: Synthetically Generated; 5000 samples and 50 features
- Loss function for a single point:
  - $f_i(w) = -y_i \log(\sigma(w^T x_i)) - (1 - y_i)\log(1 - \sigma(w^T x_i)) + \lambda/2 \|w\|^2$
- Overall objective function is the avg loss across all samples
  - $f(w) = 1/n * \sum f_i(w)$
- Two settings:
  - Convex: $\lambda = 0$, Globally lower bounded by by a linear function and if a local min exists guaranteed to be a global min
  - Strongly convex: $\lambda = 0.1$
    - Adding L2 regularization term to the loss makes the objective function strongly convex with strong convexity parameter $\mu=\lambda$. This means that the objective function curves upwards by at least λ everywhere.
  - Lipschitz Constants
    - Convex: $L = (\sum \|x_i\|^2)/4n$
    - Strongly Convex: $L = ((\sum \|x_i\|^2)/4n) + \lambda$
- All algorithms are ran for 100 epochs in both settings
  - All original paper run for fixed number of epochs, no stopping criterion
- The true optimal values for both settings are estimated using Newton's method via scipy.

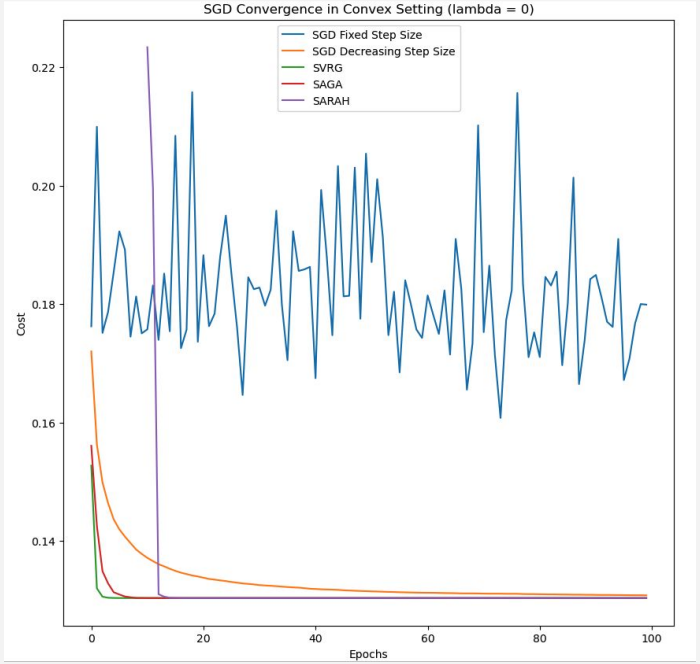
Setup continued...

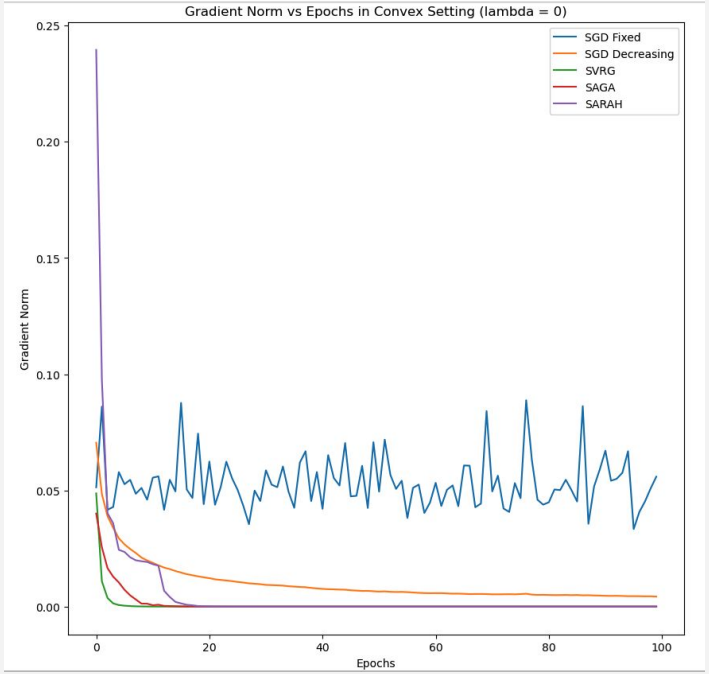
- Step sizes
  - SGD fixed step size: t = 1/L
  - SGD decreasing step size
    - Convex: t = (1/L) / √k where k is the iteration #
    - Strongly Convex: t = 1/λk where k is the iteration #
  - SVRG: t = 1/10L
  - SAGA: t = 1/3L
  - SARAH: t = 1/L

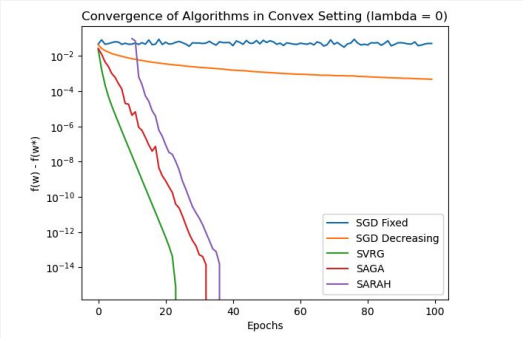
Convex Setting Results

Algorithm	Final Cost	Optimality Gap	Final Gradient Norm	Epochs to Converge
SGD Fixed	0.1799	0.0496	0.0559	None
SGD Decreasing	0.1308	0.00046	0.0043	55
SVRG	0.1303	-2.0733e-14	3.2529e-14	2
SAGA	0.1303	-2.0511e-14	1.72068e-15	4
SARAH	0.1303	-2.0678e-14	2.44018e-14	12

- True optimal cost: f(w*) = 0.13034
- SGD fixed never converges, SGD decreasing converges but slowly
- SVRG, SAGA, and SARAH all achieve an optimality gap of essentially 0
- All variance reduction methods converge faster than SGD in convex setting


SGD Convergence in Convex Setting (lambda = 0)
SGD Fixed Step Size
SGD Decreasing Step Size
SVRG
SAGA
SARAH
Cost
Epochs

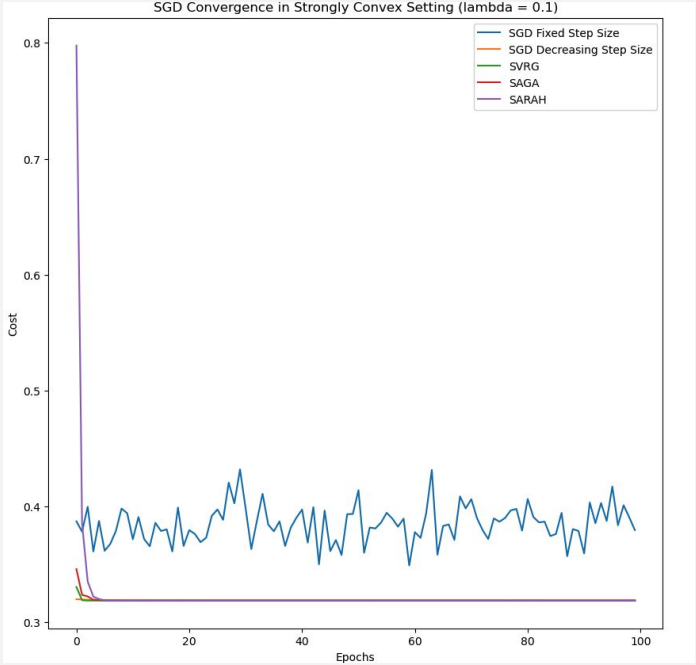

Gradient Norm vs Epochs in Convex Setting (lambda = 0)
SGD Fixed
SGD Decreasing
SVRG
SAGA
SARAH
Gradient Norm
Epochs

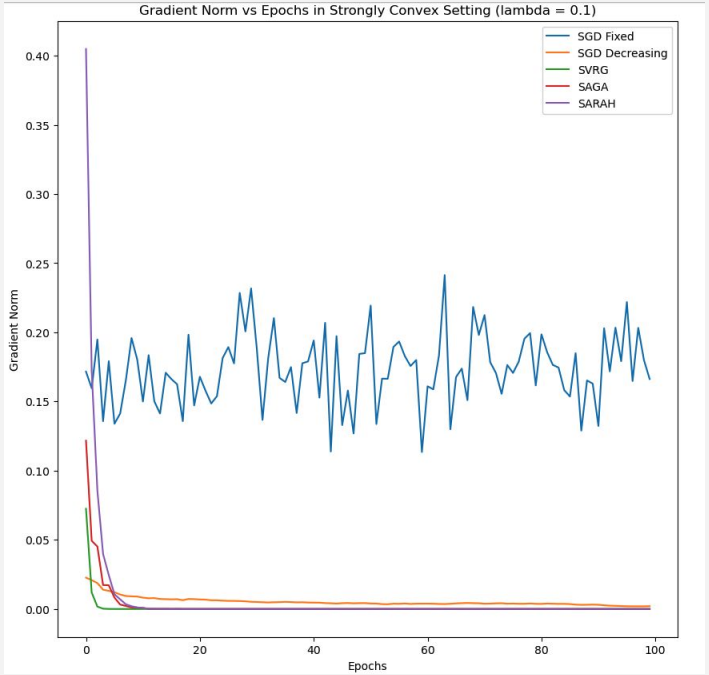

Convergence of Algorithms in Convex Setting (lambda = 0)
SGD Fixed
SGD Decreasing
SVRG
SAGA
SARAH
f(w) - f(w*)
Epochs

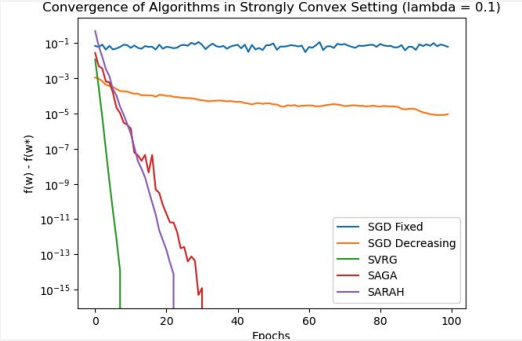
Strongly Convex Setting Results

Algorithm	Final Cost	Optimality Gap	Final Gradient Norm	Epochs to Converge
SGD Fixed	0.3796	0.0608	0.1663	None
SGD Decreasing	0.3188	9.0297e-06	0.0020	1
SVRG	0.3188	-2.387e-15	7.68429e-16	1
SAGA	0.3188	-2.276e-15	2.82543e-15	3
SARAH	0.3188	-2.276e-15	3.5532e-14	5

- True optimal cost: f(w*) = 0.3188
- SGD fixed never converges, SGD decreasing converges in 1 epoch faster than the convex setting because strong convexity gives better curvature
- SGD decreasing, SVRG, SAGA, SARAH all achieve an optimality gap of essentially 0
- All variance reduction methods converge faster than SGD in strongly convex setting


SGD Convergence in Strongly Convex Setting (lambda = 0.1)
SGD Fixed Step Size
SGD Decreasing Step Size
SVRG
SAGA
SARAH
Cost
Epochs

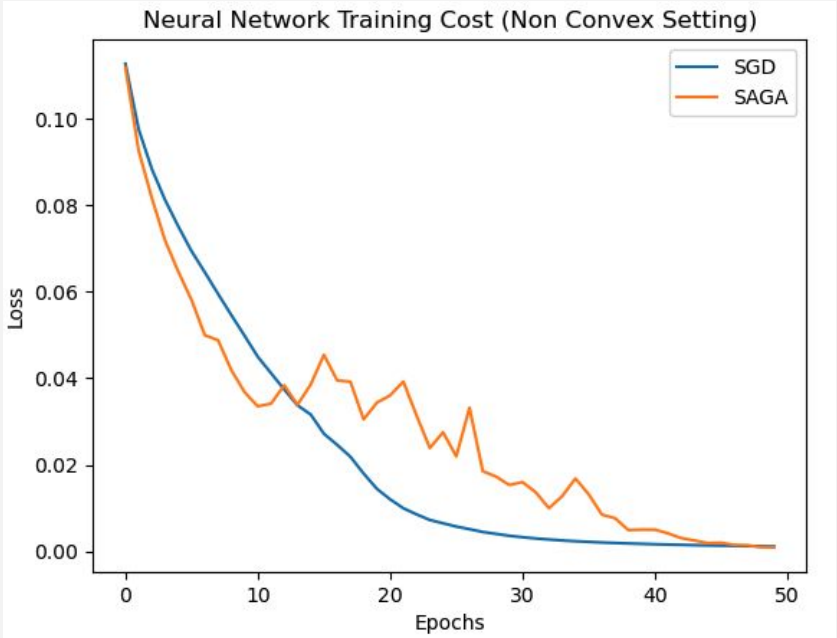

Gradient Norm vs Epochs in Strongly Convex Setting (lambda = 0.1)
SGD Fixed
SGD Decreasing
SVRG
SAGA
SARAH
Gradient Norm
Epochs

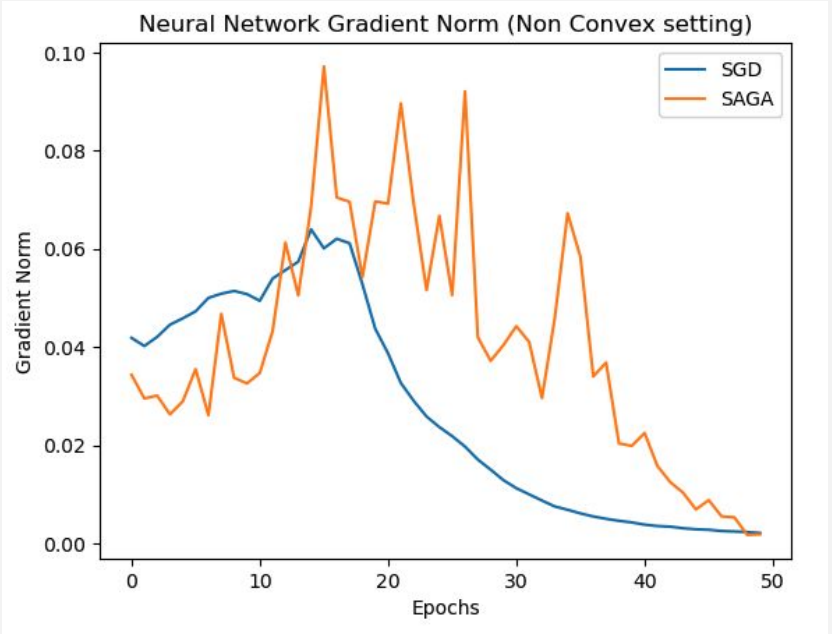

Convergence of Algorithms in Strongly Convex Setting (lambda = 0.1)
SGD Fixed
SGD Decreasing
SVRG
SAGA
SARAH
f(w) - f(w*)
Epochs

Extension of SGD and SAGA in Nonconvex Setting

- Neural network with 2 layers
  - 50 input neurons, 32 hidden layer neurons w/ Relu activation
  - 32 neurons to 1 output neuron w/ Sigmoid activation
- Optimizing same loss function as convex and strongly convex settings but the layer with non linear activation function introduces non convexity into the objective function.
- w* cannot be calculated in the non convex setting so convergence is measured using the final cost and gradient norm

Algorithm	Final Cost	Final Gradient Norm
SGD Fixed (t=0.01)	0.00114	0.00214
SAGA (t=0.01)	0.00093	0.00187


Neural Network Training Cost (Non Convex Setting)
SGD
SAGA
Loss
Epochs
0.00
0.02
0.04
0.06
0.08
0.10
0
10
20
30
40
50


Neural Network Gradient Norm (Non Convex setting)
SGD
SAGA
Gradient Norm
Epochs
0.00
0.02
0.04
0.06
0.08
0.10
0
10
20
30
40
50